

Topic Of Case Study:

Air Traffic Control system

Analysis of linked-list based data structure for ATC management

Problem Statement

What is an ATC system

- Manages and control air traffic to ensure safety and efficiency.
- Requires real-time management of airports and flights.

Objective:

- To analyze a C++ implementation of a simplified ATC system.
- To understand how nested data structures (linked lists) can model this real-world problem.

Core Functionality:

- Adding and removing airports.
- Managing flights (add, delete, update).
- Displaying flight information.
- Data Persistence (using file handling)
- Memory management

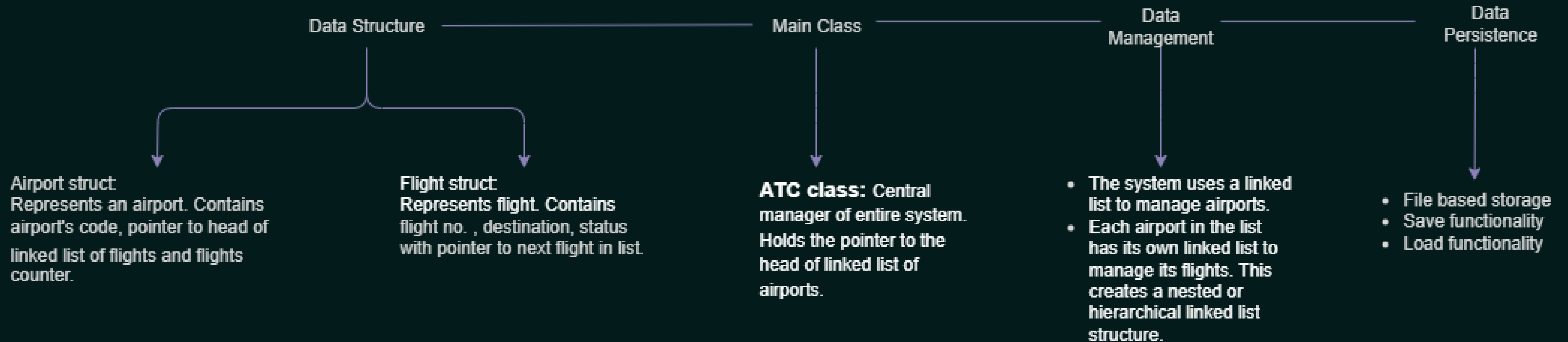
Key Features



- Airport Management: The system allows for the addition and removal of airports.
- Flight Management: Users can add new flights, delete existing ones, and update a flight's destination.
- Flight Limits: Each airport has a set limit and the system prevents adding more flights once this limit is reached.
- Data Display: The system can display a comprehensive overview of all airports and their associated flights.
- Data Persistence: All airport and flight data can be saved to a file, and then loaded back into the system later.



System Architecture

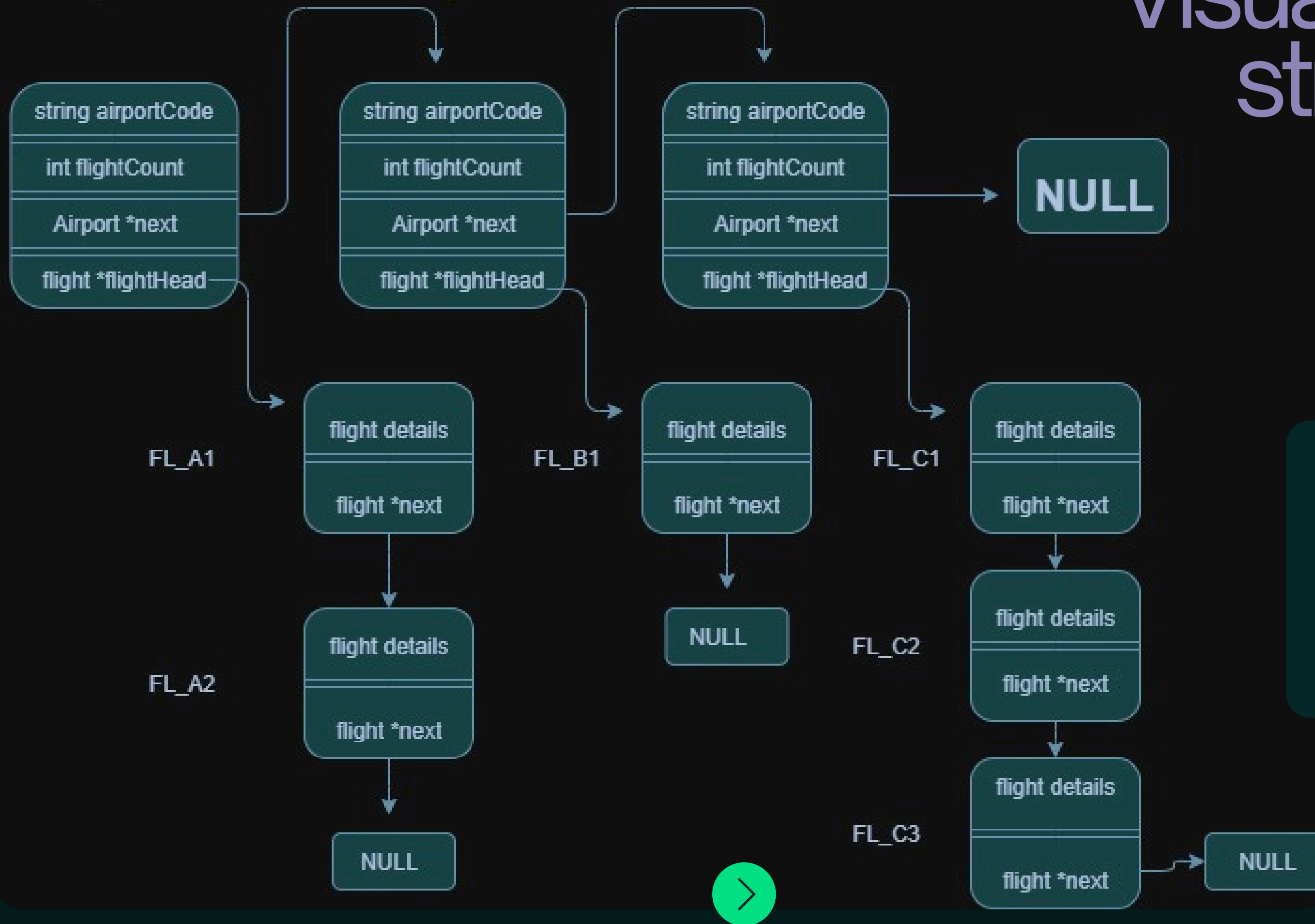


Ap1

Ap2

Ap3

Visualization of structure



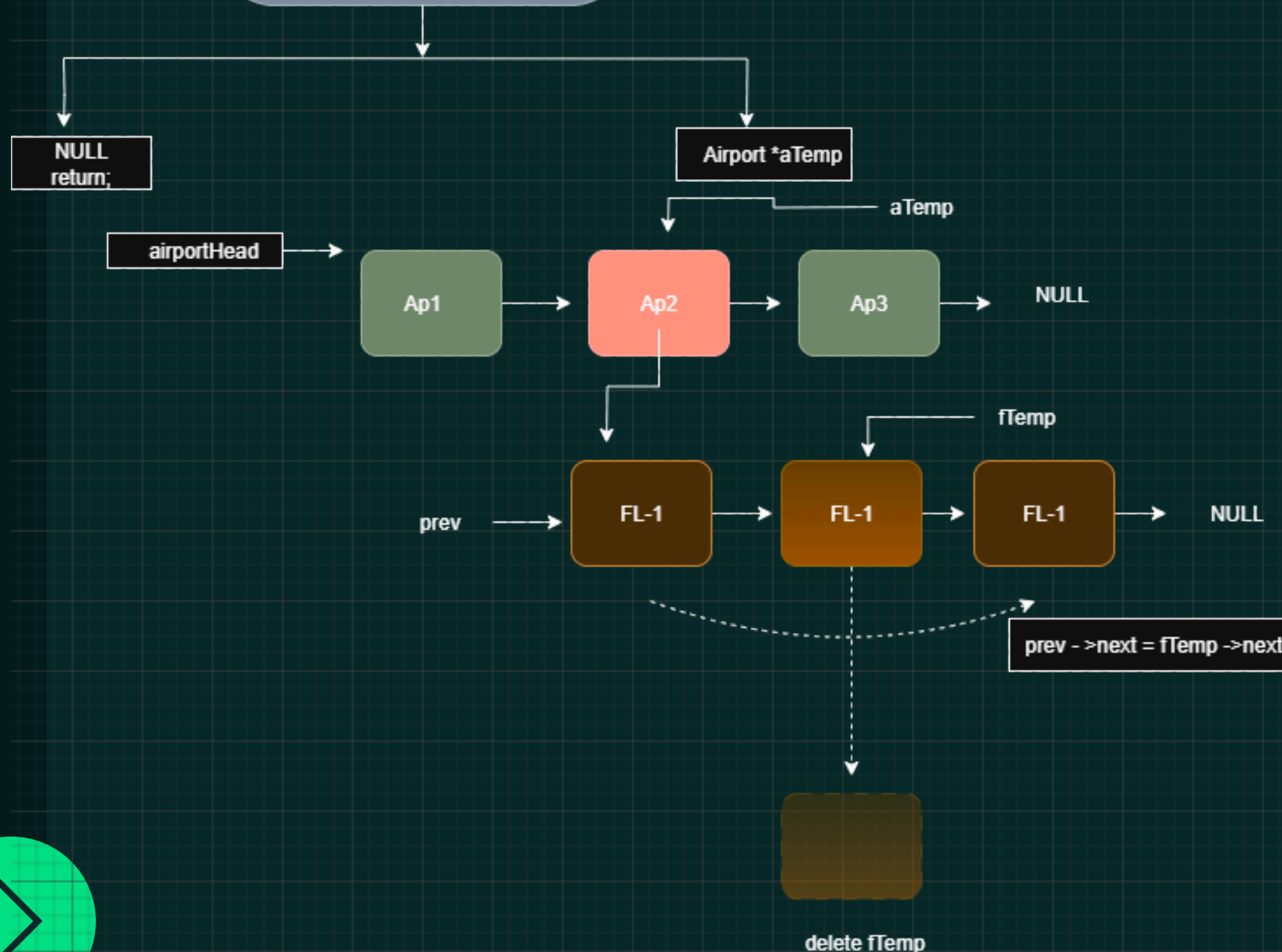
Flight details:

- Flight number
- Destination
- Status

Deletion of flights

Airport* findAirport (string code)

- Travers entire linked list of airports node by node.
- Compares input code with airportCode.
- If match found, returns pointer to that airportnode, otherwise returns null.



Time Complexity Breakdown:

- This process involves two sequential traversals:
 - 1. Airport Search (findAirport(code))**
- Operation: Traverse the linked list of airports to find a match.

Complexity:

- Worst-case: You scan all n airports $\rightarrow O(n)$
- Best-case: Match found early $\rightarrow O(1)$

2. Flight Deallocation (within matched airport)

- Operation: Traverse the flight list of the matched airport to find and delete a specific flight node.
- Complexity:
- Worst-case: You scan all m flights $\rightarrow O(m)$
- Best-case: Match found early $\rightarrow O(1)$

Combined Complexity

- In the worst case, you search all airports and all flights:
- Total Time Complexity: $O(n \times m)$
- where:
- n = number of airports
- m = number of flights in the matched airport



Deletion of airport node

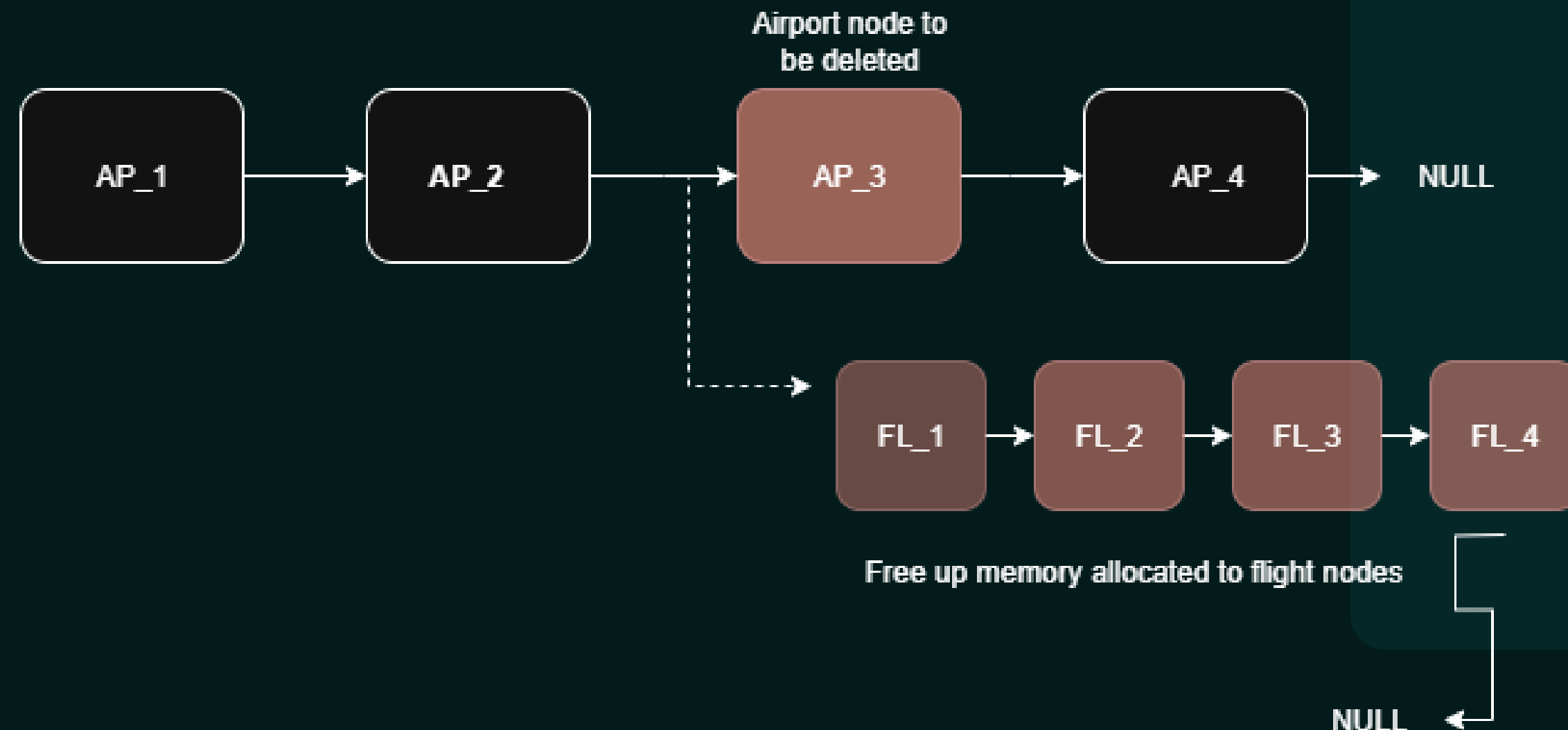
- When deleting an airport node, it is necessary to delete all the flight nodes associated with that airport first rather than just deleting the airport node alone because:

1. Avoid memory leaks: Where allocated memory is lost and cannot be reclaimed.

2. Proper Cleanup of Nested Data:
Deleting the airport node alone only frees memory for that node; the flight nodes remains allocated.

3. Pointer integrity :
Airport node will point to invalid memory after airport deletion, leading undefined behavior.

4. Maintains program stability and prevents resource wastage.



Time Complexities:

1. Airport Search (findAirport)

- Traverses airport linked list to find the matching airport.
- Worst-case: $O(n)$ (where n is the number of airports).
- Best-case: $O(1)$ if the airport is found at the head.

•

2. Flight Search within the Airport

- Traverses the flights linked list to find the matching flight by flight number.
- Worst-case: $O(m)$ (where m is the number of flights in that airport).
- Best-case: $O(1)$ if the flight is at the head.

3. Update Operation

- Once the flight node is located, the update operation (changing destination) is $O(1)$.

Clarification on $O(n \times m)$

The combined complexity is not $O(n \times m)$ since flights are traversed only after the airport is found.

$O(n \times m)$ would imply for each airport, all flights are scanned, which is not the case here.

The algorithm searches for one airport n , then searches flights m within that single airport.

Advantages of nested linked list in ATC system

Isolation of Operations:

Operations on flights within one airport do not impact flights at other airports, reducing the risk of unintended side effects and facilitating targeted updates.

Modular Expansion:

The structure can be easily extended—new data (e.g., runways, staff) can be added to the airport nodes or flight details to flight nodes with minimal disruption.

Dynamic and Flexible Structure:

Airports and flights can be created and deleted at any time, with no need to reallocate memory or resize arrays. This allows the system to scale up or down as needed.

Efficient Management of Related Data:

All flights related to a particular airport are grouped together, making it simple to add, remove, or update flights for a specific airport without affecting others.



File handling

The logic for file handling in ATC System code focuses on **saving** and **restoring** all airports and their flights so that no information is lost when the program exits and restarts.

Saving the data into the file:

- Each airport's data is saved to the file with a special marker ("AIRPORT_START") before its information, making it easy to identify when a new airport's data begins.
- For every flight under an airport, another marker (like "FLIGHTS_START") is used before saving the flight's details.
- Markers and consistent ordering allow the program, when loading, to clearly distinguish between airports and flights, and to reconstruct the relationships correctly.

Loading the data from the file:

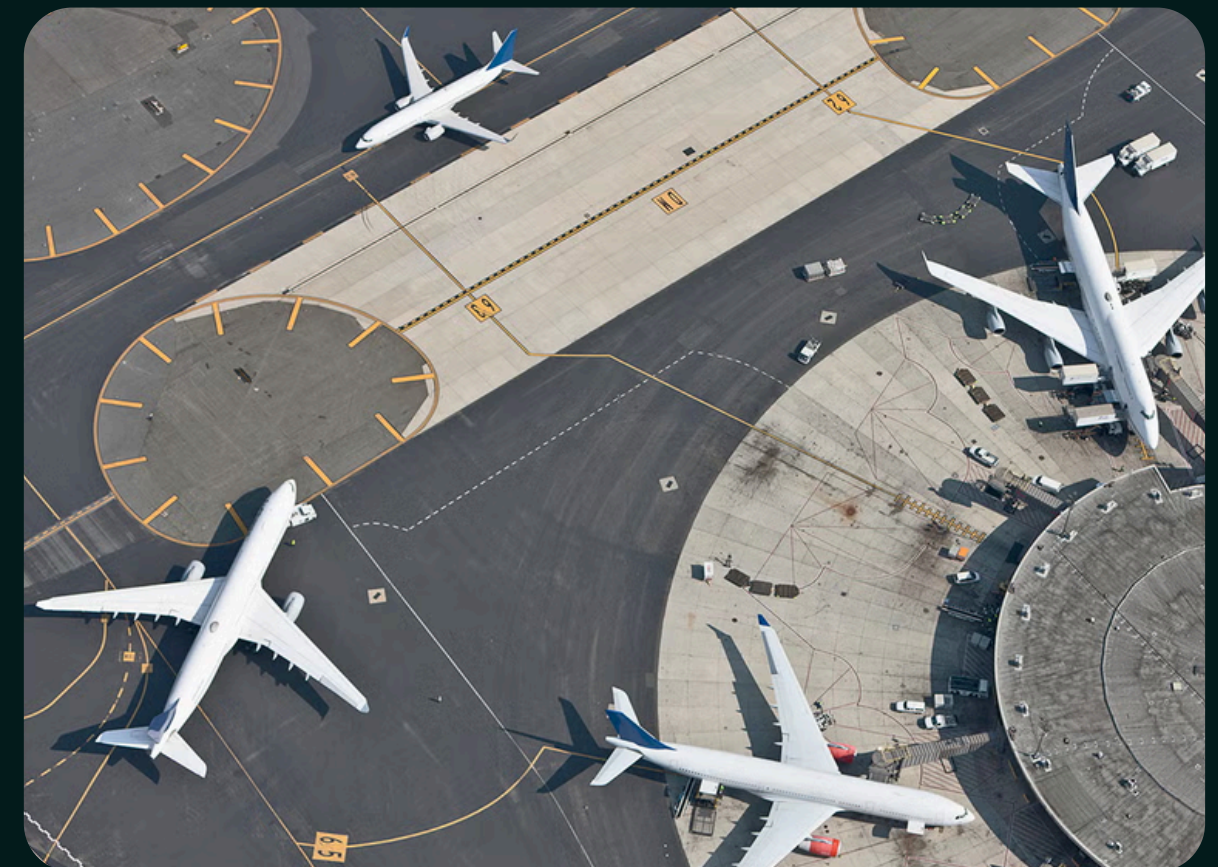
- Before loading data from the file, the program first deallocates (deletes) any previously available structure in memory.
- By clearing the existing linked lists of airports and flights, the system can safely recreate the newest structure exactly as stored in the file.
- Program reads the file in order, recognizing these marks (delimiters) to correctly rebuild the same structure in memory

This ensures we can always resume the program with the same data, making your system persistent and reliable.



Conclusion

Nested linked lists, as demonstrated in this ATC system, provide a practical, flexible, and scalable data structure ideal for managing air traffic. Their ability to dynamically manage airport and flight data, optimize memory usage, and support real-time updates makes them a valuable asset in improving air traffic management and IT operations.



Thank You

Presented by:

- | | | |
|---------------------|---|-----------|
| 1. Danish Sayyad | - | 124B1B100 |
| 2. Khushi Dudhalkar | - | 124B1B102 |
| 3. Diksha Kate | - | 124B1B108 |
| 4. Piyush Patel | - | 124B1B112 |

Course : Data Structures

Course Teacher : Prof. Ashwini Matange

